# Programovací jazyky: struktura, jména a typy

## Druhý blok kurzu: teorie kapitol 4–6

Karel Šafr

Bloková výuka

# Tři otázky o programovacím jazyce

1. Jak poznáme strukturu zápisu?
   Proč se v `2 + 3 * 4` nejprve spojuje `3 * 4`?
2. Co označuje použité jméno?
   Které `x` přečteme, když se toto jméno objeví na více místech?
3. Jaké hodnoty a operace jsou přípustné?
   Je `3 + "4"` součet, spojení textu, nebo chyba?

U každé otázky oddělíme pravidlo jazyka, jeho realizaci a výsledek konkrétního běhu.

Pojmy vysvětlíme zde; předchozí četbu nepředpokládáme. Mini ilustruje teorii, resolver je samostatný domácí úkol.

# Zápis programu a jeho význam

**Syntaxe** určuje přípustný zápis. **Sémantika** určuje jeho význam. **Výraz**, například `x + 1`, určuje hodnotu.

| | |
|---|---|
| Syntaxe | `x + 1` má tvar součtu; `x + * 1` v našem modelu ne. |
| Statická pravidla | Posuzují program před vykonáním: je jméno `x` zavedeno? |
| Dynamická pravidla | Popisují vykonání: je-li v `x` číslo 4, výraz `x + 1` dá 5. |

**Deklarace** zavádí jméno, např. `var x = 4;` v Mini. **Stav programu** zahrnuje aktuálně uložené hodnoty.

Správný tvar ještě nezaručuje přípustná jména ani správný výsledek úlohy.

# Jazyk a jeho implementace

**Implementace** je nástroj, který pravidla jazyka uskutečňuje.

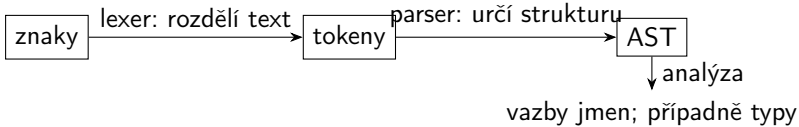
- **Interpret** provádí kroky programu z jeho zpracované podoby.
- **Překladač** vytváří jiný program: například instrukce procesoru nebo zdrojový text C++.
- **Virtuální stroj** je softwarový vykonavatel vlastní sady instrukcí; jejich zápis bývá bajtkód.

Překlad a interpretace se mohou kombinovat: zdroj se nejdříve přeloží do bajtkódu a ten se potom vykonává.

Pravidlo, že `2 + 3 * 4` dá 14, samo neurčuje použitou implementační cestu.

# Fáze pracují s různými informacemi

Model: `x = x + 1;` s dříve deklarovaným `x`.
**Token** je rozpoznaná jednotka zápisu. **AST** je abstraktní syntaktický strom: zachycuje stavbu programu.




- **Lexer** pozná jména, čísla a značky; **parser** z nich složí například přiřazení se součtem.
- **Resolver** hledá deklarace jmen. Typová analýza zkoumá, zda operace přijímá dané druhy hodnot.
- Teprve vykonání pracuje s aktuální hodnotou: při $x = 4$ zapíše do `x` číslo 5.

# Lexém, token, identifikátor a literál

V zápisu `cena = 12 + 3;` je **lexém** konkrétní úsek textu. **Token** tento úsek označí druhem a případnými dalšími údaji.

| Lexém | Druh tokenu | Příklad údaje |
|---|---|---|
| `cena` | identifikátor | text jména |
| `=` | přiřazení | pozice ve zdroji |
| `12` | číselný literál | hodnota 12 |
| `+` | plus | pozice ve zdroji |
| `3` | číselný literál | hodnota 3 |
| `;` | středník | pozice ve zdroji |

**Identifikátor** je zápis jména, např. `cena`. **Literál** zapisuje hodnotu přímo, např. `12` nebo text `"ahoj"`.

Lexer pozná tvar jména; ještě nehledá jeho deklaraci.

# Pravidlo pro zápis identifikátoru

**Regulární výraz** je vzor sestavený z voleb a opakování znaků.
Zvolme písmena `A-Z`, `a-z` z ASCII a podtržítko; až na dalších pozicích dovolíme také číslice.

```
[A-Za-z_][A-Za-z0-9_]*
```

| | |
|---|---|
| `x`, `for2`, `_pocet` | pravidlu vyhovují |
| `2x`, `cena-kusu` | celé nejsou jedním identifikátorem |

- `[...]` vybírá jeden znak; `*` dovoluje nula či více opakování předchozí skupiny.
- ASCII zde znamená základní znaky bez české diakritiky.
- UTF-8 je kódování textového souboru; samo nepovoluje česká písmena ve jménech jazyka.

# Stejné pravidlo jako konečný automat

**Konečný automat** si pamatuje jeden z konečného počtu stavů. Další znak určí přechod do nového stavu.

| Stav | Další znak | Nový stav |
|---|---|---|
| $q_0$: začátek | písmeno nebo _ | $q_1$ |
| $q_1$: alespoň první znak | písmeno, číslice, _ | $q_1$ |
| oba stavy | jiný znak | zamítnutí kandidáta |

Přijímající stav je $q_1$. Prázdný řetězec se nepřijímá. Stopa a2: $q_0 \xrightarrow{a} q_1 \xrightarrow{2} q_1$. Lexer

(též scanner) celého programu navíc určuje hranice tokenů: u a2; ponechá středník dalšímu kroku.

# Klíčové slovo se pozná z celého lexému

**Rezervované klíčové slovo** má zvláštní úlohu v jazyce a nelze je použít jako běžné jméno. Zde jde o `for`.

1. Dočteme celý lexém identifikátorového tvaru.
2. Porovnáme jej s tabulkou rezervovaných slov.
3. Vrátíme klíčové slovo nebo identifikátor.

| | |
|---|---|
| `for` | klíčové slovo |
| `for2` | jeden identifikátor |
| `For` | identifikátor, pokud rozlišujeme velikost písmen |

Rozdělení `for2` na `for` a `2` by změnilo program.

# Lexer: kurzor a hranice tokenu

Vstup: `count12+3`. **Kurzor** zde znamená pozici dalšího znaku ke zpracování; platí předchozí pravidlo jmen.

| Krok | Rozpoznaný text | Následující znak |
|---|---|---|
| Začátek jména | c | o |
| Další písmena | `count` | 1 |
| Číslice uvnitř jména | `count12` | + |
| Vrácení identifikátoru | `IDENT(count12)` | stále + |
| Další tokeny | `PLUS`, `NUMBER(3)` | konec |

- Prohlédnuté + se nestalo součástí identifikátoru.
- `x+12` a `x + 12` mají stejné tokeny.
- `xy` a `x y` mají různé tokeny: mezery nelze předem smazat.

# Gramatika určuje vztahy mezi tokeny

**Gramatika** je soubor pravidel přípustné struktury. Zapíšeme ji v EBNF, rozšířené Backusově–Naurově formě:

```
E = T { ("+" | "-") T } ;
T = F { ("*" | "/") F } ;
F = NUMBER | IDENT | "(" E ")" ;
```

- **Neterminály** se dále rozvíjejí: `E` je výraz, `T` člen, `F` základní část výrazu.
- **Terminály** jsou výsledné druhy tokenů: `NUMBER`, `IDENT` a značky v uvozovkách.
- `|` znamená „nebo“, `{ }` nula či více opakování; závorky bez uvozovek seskupují pravidlo.

`=` zavádí pravidlo, poslední `;` je uzavírá; uvozovky vymezují skutečné tokeny programu. `2 + 3 * 4` pravidlům vyhoví; `2 + * 4` ne.

# Platné tokeny ještě netvoří platný výraz

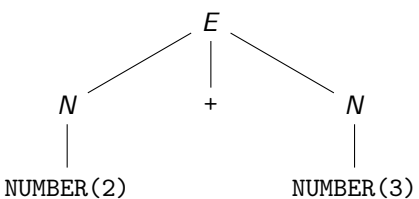
| Zdroj | Lexer | Parser podle `E/T/F` |
|---|---|---|
| `2 + 3` | Všechny tokeny zná. | Přijme celý výraz. |
| `2 + * 3` | Všechny tokeny zná. | Po + nemůže člen začít `*`. |
| `(2 + 3` | Všechny tokeny zná. | Nenajde požadované `)`. |
| `2 + 3 4` | Všechny tokeny zná. | Za správným začátkem zbývá token `4`. |

Lexer rozpoznává jednotky. Parser kontroluje jejich vztahy a úplnost konstrukce.

# Pravidla vytvářejí derivační strom

Pravidlo $E \to N + N$ čteme „$E$ rozviň na $N + N$“; dále $N \to$ `NUMBER`. $\Rightarrow$ označuje krok odvození:

$$E \Rightarrow N + N \Rightarrow \texttt{NUMBER} + N \Rightarrow \texttt{NUMBER} + \texttt{NUMBER}$$

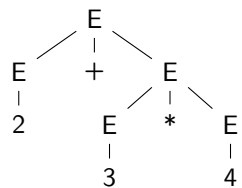



- **Kořen**: horní uzel, zde počáteční symbol $E$.
- **Hrany** spojují uzel s jeho potomky: částmi podle pravidla.
- **Listy** už nemají potomky; zleva dávají tokeny 2+3.

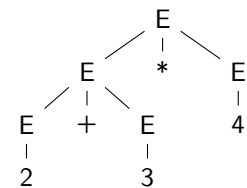
**Podstrom** je vybraný uzel se všemi svými potomky.

# Nejednoznačná gramatika dovolí dva stromy

Zkusme jiné pravidlo: $E \to E + E \mid E * E \mid \texttt{NUMBER}$.
Stejný zdroj 2+3*4, stejné listy; jiná vnitřní struktura:




$2 + (3 * 4) = 14$




$(2 + 3) * 4 = 20$

Číselné listy jsou tokeny `NUMBER`. Pravidla `E/T/F` tuto nejednoznačnost odstraní.

# Příklad: jak čteme funkci a její volání

Vlastní pseudokód: **funkce** zde pojmenovává opakovatelný postup.

```
function dvojnasobek(n):
    return n + n

vysledek = dvojnasobek(3)
```

- `n` je **parametr**: místní jméno vstupu funkce.
- `3` je **argument**: výraz ve volání; zde přímo dodává hodnotu 3.
- `return` vrátí výsledek volajícímu; zde do `vysledek` uložíme 6.

**Příkaz** `return` provede krok programu. Parserové funkce budou vracet strom: `Num(2)` značí uzel čísla; `Add(a,b)` uzel součtu a `Mul(a,b)` uzel součinu s podstromy `a`, `b`.

# Rekurzivní sestup a pohled vpřed

**Rekurzivní sestup** je jedna možná strategie parseru.
**Rekurze**: funkce použije znovu sebe, přímo nebo přes jiné funkce.

- Kategoriím `E`, `T`, `F` odpovídají `parseE`, `parseT`, `parseF`.
- **Pohled vpřed (lookahead)** prohlíží další dosud nespotřebovaný token a pomáhá zvolit pravidlo.

U `(2 + 3)` funkce `parseF`:

1. spotřebuje `(`, tedy posune pozici vstupu,
2. zavolá `parseE` pro vnitřní výraz,
3. vyžádá a spotřebuje `)`.

**Prohlédnutí tokenu vstup neposouvá; spotřeba ano.**

# Stopa parseru: první člen skončí před plus

Tokeny pro 2+3*4; **index** je jejich pořadové číslo od nuly. `EOF` označuje konec vstupu (end of file).

| `2` | `+` | `3` | `*` | `4` | `EOF` |
|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 |

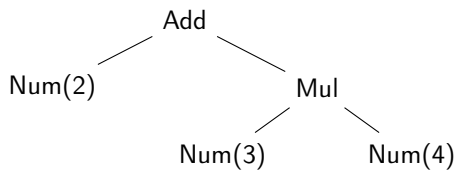
| Krok | Index poté | Pohled vpřed |
|---|---|---|
| `parseE` zavolá `parseT`, ta `parseF`. | 0 | 2 |
| `parseF` spotřebuje číslo; vytvoří `Num(2)`. | 1 | + |
| `parseT` nevidí `*` ani `/`; vrátí podstrom. | 1 | + |
| `parseE` spotřebuje +. | 2 | 3 |

Člen neodebere cizí operátor. + počká na funkci pro výraz.

# Stopa parseru: násobení vznikne uvnitř členu

Pokračujeme od indexu 2: `3 * 4 EOF`.

1. `parseT` přečte 3, potom `*` a 4; index je 5.
2. Vrátí podstrom `Mul(Num(3), Num(4))`.
3. `parseE` připojí oba členy pod uzel `Add`.




Volající ověří `EOF`. Strom je hotový; hodnota 14 vznikne až při vyhodnocení.

# Proč naivní levá rekurze nepostupuje

$$E \to E \text{ - } T \mid T$$

Naivní volba první větve volá $E$ znovu na stejné pozici:

$$E(0) \to E(0) \to E(0) \to \cdots$$

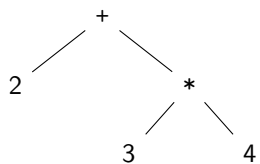
- Ani jeden token se nespotřeboval.
- Pro tento příklad lze použít $E \to T \ \{\text{- } T\}$.
- Každé opakování spotřebuje operátor - i další operand (zde číslo), nebo ohlásí chybu.

Tato úprava sama neřeší všechny možné gramatiky.

# Priorita je vidět ve stromu

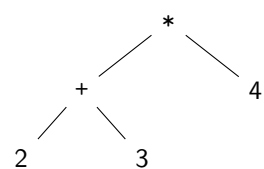
**Priorita (precedence)** určuje, který operátor váže své operandy těsněji. V `E/T/F` má násobení přednost před součtem.

`2 + 3 * 4`




Hodnota při vyhodnocení: 14.

`(2 + 3) * 4`



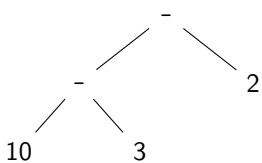

Hodnota při vyhodnocení: 20.

**Operátor** určuje operaci (+, *); **operandy** jsou její vstupy. Ve stromu jsou operandy podstromy pod operátorem.

# Asociativita mění tvar AST

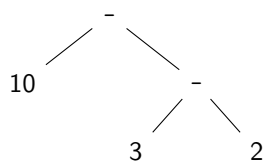
**Asociativita** určuje seskupení operátorů stejné priority. Pro tokeny `10 - 3 - 2` porovnejme dvě pravidla:

Zleva: $(10 - 3) - 2$




Výsledek 5.

Zprava: $10 - (3 - 2)$




Výsledek 9.

Náš model i Mini použijí vlevo asociativní odčítání.

Odstranění levé rekurze musí zachovat také tuto stavbu AST.

# Opakování skládá AST zleva

Pseudokód pro `E = T { ("+" | "-") T }`. `while` opakuje odsazené kroky, dokud platí podmínka:

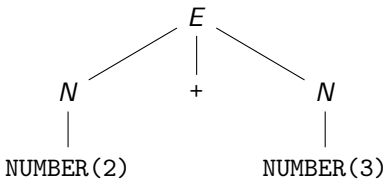
```
left = parseT()
while lookahead is PLUS or MINUS:
    op = consume()
    right = parseT()
    left = Binary(op, left, right)
return left
```

- `consume()` odebere token; `Binary` vytvoří uzel, nepočítá hodnotu.
- U `10-3-2` se první strom `Minus(10,3)` stane levým potomkem nového uzlu.
- Dostaneme `Minus(Minus(10,3),2)`, tedy $(10 - 3) - 2$.

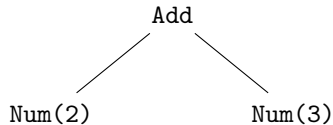
# Derivační strom a AST téhož výrazu

Pro malou gramatiku $E \to N + N$, $N \to$ `NUMBER` a zdroj 2+3:

Derivační strom




Jedna možná podoba AST




- Derivační strom zaznamenává použití pravidel gramatiky.
- AST vypouští pomocné kategorie a ponechává operaci i operandy.
- Parser může AST vytvářet přímo, bez uložení druhého stromu.

# Mezireprezentace a přepis konstrukcí

**Mezireprezentace (IR)** je podoba programu pro další fáze překladu, například AST. **Lowering** přepisuje konstrukce na jednodušší se zachováním významu. Vlastní

pseudokód: smyčka má vypsat postupně 0, 1, 2.

Začátek; test; krok:

```
for i = 0; i < 3; i = i + 1:
    print i
```

Test a opakování:

```
i = 0
while i < 3:
    print i
    i = i + 1
```

Vpravo testujeme podmínku před každým průchodem. Obě varianty zde používají stejné `i`; po skončení je 3.

Přepis obecně musí zachovat také dostupnost jmen a pořadí účinků.

# Chybu zařazujeme podle porušeného pravidla

Model se středníky a deklaracemi. `print` znamená „vypiš“; **řetězec** je textová hodnota, zde `"x"`. + nemíchá text s číslem:

| | |
|---|---|
| `print @;` | Lexer: nepřípustný znak. |
| `print 1` | Parser: chybí povinný středník. |
| `print x;` | Kontrola vazeb: `x` nemá deklaraci. |
| `print "x" + 1;` | Typová chyba; čas odhalení určuje typový systém. |

Pozice hlášení často označuje místo zjištění chyby; příčina může ležet dříve.

# Stejné kategorie tokenů, jiné statické otázky

Model se statickou kontrolou: `x` je celé číslo, `flag` je bool (pravdivostní hodnota `true`/`false`), `z` není deklarováno. Operátor `+` přijímá jen čísla.

| Příkaz | Výsledek statické analýzy |
|---|---|
| `x = x + 1;` | Jména i typy odpovídají pravidlům. |
| `x = z + 1;` | Nelze najít deklaraci `z`. |
| `x = flag + 1;` | Jméno existuje, ale typ operandu je nesprávný. |

Všechny mají vzor `IDENT = IDENT + NUMBER ;`.

- Resolver potřebuje jména a deklarace; typová analýza také typy.
- Žádný z těchto kroků ještě nemusí znát aktuální hodnotu `x`.

# Krátká ukázka Mini: od struktury k výsledku

K04-01, úplný program pro hotové Mini:

```
fun main() {
  var x = 1;
  var y = 10;
  var z = x + y;
  print z;
}
```

- `fun main()` zavádí vstupní funkci bez parametrů.
- `{ }` vymezují její tělo, `var` zavádí proměnnou.
- `print` vypíše hodnotu; `;` ukončuje příkaz.

`tokens`: jednotky; `ast`: struktura; `check`: vazby jmen.
`cpp`: zdroj C++; `run`: přeložení a běh, výstup 11.

**Běhové prostředí (runtime)** poskytuje operace nad skutečnými hodnotami, zde sčítání a výpis.

# Úloha: tokeny, strom, vazby a hodnoty

Použijte gramatiku E/T/F. `stmt` značí příkaz; první řádek je pravidlo, druhý je zdroj k rozboru:

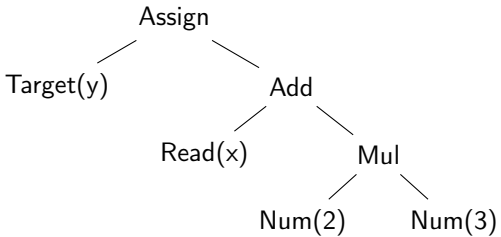
```
stmt = IDENT "=" E ";" ;
y = x + 2 * 3;
```

Model: `x` a `y` jsou deklarovaná celá čísla; na začátku $x = 4$, $y = 9$. Operace mají obvyklý číselný význam.

1. Zapište tokeny včetně jejich lexémů.
2. Nakreslete AST se samostatným cílem přiřazení.
3. Která fáze potřebuje deklaraci `x`? Která jeho hodnotu?
4. Co bude v `y`? Změní se AST, pokud začneme s $x = 9$?

# Řešení: struktura zůstává, hodnota závisí na stavu

Tokeny: `IDENT(y)`, `=`, `IDENT(x)`, `+`, `NUMBER(2)`, `*`, `NUMBER(3)`, `;`
`Assign`: přiřaď; `Target`: cíl zápisu; `Read`: přečti hodnotu.




- Lexer rozpozná lexémy.
- Parser určí tento strom.
- Resolver spojí `x` a `y` s deklaracemi.
- Vyhodnocení čte hodnotu `x` a zapisuje do `y`.

Pro $x = 4$ vznikne $y = 10$; pro $x = 9$ vznikne $y = 15$.
V obou případech se `x` nezmění a AST má stejný tvar.

# Jméno, místo a hodnota

V modelu, kde příkazy mění stav programu, oddělíme tři otázky:

| | |
|---|---|
| Jméno | Jak proměnnou označujeme ve zdrojovém programu? |
| Místo | Které měnitelné úložiště proměnná reprezentuje? |
| Hodnota | Co v něm právě je? |

Po inicializaci `x = 10` a následném přiřazení `x = 11` se mění obsah téhož místa.
**Objekt** zde znamená konkrétní data existující za běhu; nemusí jít o instanci třídy. Místo je model úložiště, nikoli nutně fyzická adresa.

# Deklarace, inicializace a přiřazení

Pseudokód s explicitní deklarací:

```
var pocet = 3;
pocet = pocet + 1;
```

1. Deklarace zavádí jméno a pravidla jeho použití.
2. Inicializace stanoví první hodnotu, zde 3; **inicializátor** je výraz, který ji určuje.
3. Přiřazení změní hodnotu existující proměnné, zde na 4.

Jazyky mohou tyto kroky spojovat nebo oddělovat různě.

# Vazba a okamžik jejího vzniku

**Vazba (binding)** je spojení dvou věcí, například použití jména s deklarací. Vždy určujme, *co s čím* spojujeme.

- Statická vazba vzniká před během a během sledovaného vykonání se nemění.
- Dynamická vazba vzniká nebo se může měnit za běhu.

| | |
|---|---|
| Význam operátoru | Může určit návrh jazyka. |
| Reprezentace čísla | Může určit volba implementace. |
| Použití jména a deklarace | Může určit statická analýza. |
| Proměnná a aktuální hodnota | Určí inicializace a přiřazení za běhu. |

Statická vazba jména nevylučuje měnící se hodnotu proměnné.

# Pojmenovaná konstanta a neměnná vazba

Konstanta uchovává po inicializaci tutéž hodnotu. Ta nemusí být známá již při překladu. V našem pseudojazyce `let` zakáže nové přiřazení. Seznam `[1]` je objekt s jednou položkou; **reference** je hodnota, která k objektu poskytuje přístup.

```
let limit = read_number();
let data = [1];
```

| | |
|---|---|
| `limit = 100;` | Zakázáno: mění jednou určenou hodnotu. |
| `data = [2];` | Zakázáno: nahrazuje referenci jinou. |
| `data.append(2);` | Povoleno v tomto modelu: mění seznam, ne referenci. |

`read_number()` načte číslo; `append(2)` přidá položku. Obsah seznamu pak bude `[1, 2]`, reference zůstane stejná.

# Lexikální rozsah určuje struktura textu

**Rozsah platnosti (scope)** určuje, kde lze dané jméno použít. Blok je skupina příkazů, v ukázkách uzavřená do `{ }`.

$$\text{vnitřní blok} \longrightarrow \text{obalující blok} \longrightarrow \text{další vnější oblast.}$$

- Použití jména se přiřadí k deklaraci podle textového vnoření.
- **Stínění**: bližší stejnojmenná deklarace skryje vnější jméno.
- Konkrétní pravidlo určí, odkdy je deklarace dostupná.
- Řetězec volajících funkcí sám tuto vazbu neurčuje.

V Mini smí vnořený blok jméno zastínit. Dvě deklarace téhož jména v jednom bloku znamenají **duplicitu** a chybu. Přiřazení novou deklaraci nezavádí.

# Krátká ukázka Mini: dvě různá x

K05-01, pouze zkrácené zalomení původního programu:

```
fun main() {
  var x = 10; print x;
  { var x = 20; print x;
    x = x + 1; print x; }
  print x;
}
```

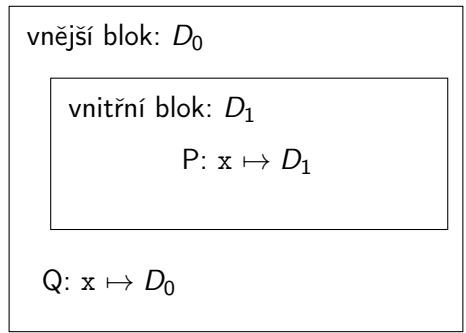
Výstup: 10, 20, 21, 10.

- Vnitřní přiřazení mění vnitřní proměnnou.
- Vnější proměnná během zastínění nezanikla.

# Prostředí v bodech P a Q

K05-01: $D_0, D_1$ označují dvě deklarace. $x \mapsto D_1$ čteme: „použití x patří k deklaraci $D_1$“.

```
fun main() {
  var x = 10;     // D0
  {
    var x = 20;   // D1
    x = x + 1;
    print x;      // P
  }
  print x;        // Q
}
```




**Prostředí** zde je soubor dostupných vazeb jmen na deklarace. Hodnoty v tomto běhu jsou v P: 21, v Q: 10.

# Od kterého místa je jméno dostupné?

Jazyk musí určit pravidlo viditelnosti deklarace:

- od určitého bodu v pořadí příkazů,
- v celém vymezeném úseku programu,
- případně s další podmínkou před použitím.

Otázka `var x = x + 1;` proto vyžaduje dvě odpovědi:

1. Ke které deklaraci patří pravé `x`?
2. Je dovoleno ji v tomto okamžiku číst?

Samotný pojem lexikálního rozsahu úplnou odpověď nedává.

# Dynamický rozsah: jiné pravidlo hledání

K05-05, pseudokód: **globální** jméno je mimo funkce, **lokální** uvnitř funkce. **Aktivace** znamená jedno probíhající volání.

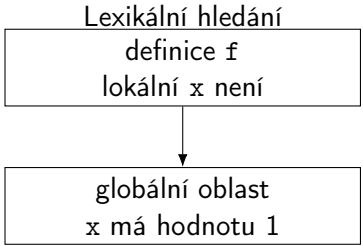
```
var x = 1;
function f() { print x; }
function g() { var x = 2; f(); }
g();
```

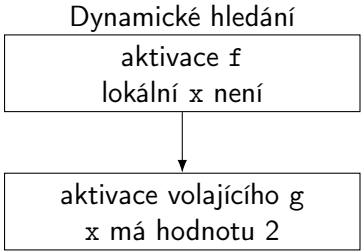
- Lexikální hledání: z definice `f` do textově vnější oblasti.
- Dynamické hledání: z aktivace `f` k právě volající `g`.

Jaký výpis dostaneme při každém z těchto pravidel?

# Stejný řetězec volání, jiné vazby

K05-05: obě funkce jsou definovány globálně; g právě zavolala `f`.




Šipky ukazují další místo hledání jména, nikoli kopírování hodnot.

- Dynamický rozsah umožňuje vliv prostředí volajícího.
- Tady hledáme proměnnou. Typování, tedy pravidla použití druhů hodnot, probereme zvlášť.

# Jak číst ukázku C++ a Bashe

| Zápis | Význam v následujícím příkladu |
|---|---|
| C++ `#include <iostream>` | Zpřístupní prostředky pro vstup a výstup. |
| C++ `int x = 1;` | Zavede celočíselnou proměnnou s první hodnotou 1. |
| C++ `void f()` | Funkce `f` bez argumentů a bez vracené hodnoty. |
| C++ `std::cout << x` | Vypíše hodnotu `x`; `'\n'` ukončí řádek. |
| Bash `local x=2` | Zavede lokální proměnnou. U přiřazení nejsou mezery kolem =. |
| Bash `echo "$x"` | Přečte a vypíše hodnotu `x`. |
| Bash `f` | Zavolá funkci; při volání se nepíší závorky. |

**Podstatný rozdíl:** kde je `f` napsaná (C++), nebo kdo ji právě zavolal (Bash).

# C++ a Bash: které x přečte volaná funkce?

B2-SCOPE: stejné hodnoty a pořadí volání, odlišné pravidlo hledání.

**C++17: lexikální hledání**

```
#include <iostream>
int x = 1;
void f() { std::cout << x << '\n'; }
void g() {
  int x = 2;
  f();
  std::cout << x << '\n';
}
int main() { g(); f(); }
```

Výstup po řádcích: `1`, `2`, `1`.

**Bash: dynamické hledání**

```
x=1
f() { echo "$x"; }
g() {
  local x=2
  f
  echo "$x"
}
g
f
```

Výstup po řádcích: `2`, `2`, `1`.

- První výpis dělá `f`: v C++ čte globální `x`, v Bashi lokální `x` volající `g`.
- Druhý výpis čte přímo `g`; třetí vzniká až po jejím návratu.

# Jmenné prostory oddělují účely jmen

- **Jmenný prostor** je skupina jmen, ve které se vyhledává. Jazyk může oddělit například jména funkcí a proměnných.
- Stejný zápis ve dvou takových skupinách pak nemusí znamenat konflikt.
- To, zda právě voláme funkci, nebo čteme proměnnou, může určit skupinu pro hledání.

Příklad návrhové volby: jméno typu a jméno proměnné mohou být hledána odděleně.
Příklad v Mini: volání `f()` hledá funkci, výraz `f` proměnnou. Rozsah navíc určí, ve které oblasti je jméno dostupné.

# Rozsah jména a životnost objektu

| Lexikální rozsah | Životnost |
|---|---|
| Oblast zdrojového programu. | Interval během vykonávání. |
| Kde se jméno může vztahovat k deklaraci? | Kdy objekt existuje a lze jej platně používat? |

- Zastíněné jméno může označovat stále živý objekt.
- Objekt může přežít blok, v němž vznikl.
- Jedna deklarace může při více voláních vytvořit více objektů.

# Jedna deklarace, více aktivací

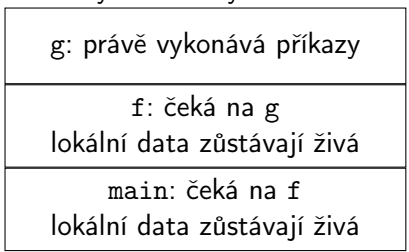
Pseudokód:

```
function f(a) {
  var x = a;
  return x;
}
f(10); f(20);
```

- V textu je jediná deklarace lokálního `x`.
- Každé volání má vlastní aktivaci a vlastní lokální objekt.
- Druhé volání nepoužívá původní lokální hodnotu 10.

# Současně živé aktivace

Model běžných vnořených volání: `main` zavolá `f`, ta zavolá `g`.




1. Vrátí se `g`; pokračuje `f`.
2. Vrátí se `f`; pokračuje `main`.

Aktivace je živá od vstupu do funkce do jejího návratu. Pozastavený volající nezanikl.

Obrázek modeluje pořadí volání a návratů. Neslibuje fyzické umístění všech dat na zásobníku.

# Doba uložení a paměť pro objekt

Doba uložení určuje, jak dlouho je k dispozici paměť pro objekt.

| | |
|---|---|
| Statická | Úložiště trvá po dobu programu. |
| Automatická | Typicky souvisí s aktivací bloku nebo funkce. |
| Dynamická | Vzniká na vyžádání; konec závisí na pravidlech správy paměti. |

- **Alokace** přidělí úložiště; inicializace mu dá první obsah.
- **Zásobník** spravuje paměť podle vnořených volání; **halda** umožňuje jinou životnost přidělených objektů.
- Jde o způsoby realizace. Přesná pravidla určuje jazyk a implementace.

# C++: lokální jméno, hodnota mezi voláními

B2-T01, C++: dvě volání mají sdílet jedno počítadlo.

```
int dalsi() {
  static int n = 0;
  return ++n;
}
// Dve postupna volani dalsi(): 1, potom 2.
```

- `int` je celé číslo; `++n` zvýší `n` o 1 a dá novou hodnotu.
- `static` zde zachová objekt mezi voláními; jméno `n` je lokální.
- Inicializace tohoto lokálu proběhne jednou; hodnota se zachová.

# Prostředí, tabulka symbolů a běhový stav

Vraťme se k bodu P z K05-01, za vnitřním `x = x + 1;`.

| Pohled | Co zaznamenává |
|---|---|
| Prostředí v bodě P | Pro jméno `x` se vybere deklarace $D_1$. |
| Tabulka symbolů | Záznamy analyzátoru o deklaracích: jejich označení, oblast a další údaje. Mohou zahrnovat i zastíněné $D_0$. |
| Běhový stav | V této aktivaci má objekt podle $D_0$ hodnotu 10 a objekt podle $D_1$ hodnotu 21. |

- Určit deklaraci neznamená vypočítat její inicializátor.
- Seznam vnořených rozsahů v analyzátoru není seznam běžících volání.

# Objekt může přežít jméno svého tvůrce

Model s referencemi: objekt je **dosažitelný**, pokud k němu program ještě má cestu přes dostupné odkazy.

1. Vnější proměnná `vysledek` zatím nic neoznačuje.
2. Vnitřní blok vytvoří objekt a lokální odkaz `docasny`.
3. Odkaz na objekt uloží také do `vysledek`.
4. Blok skončí; lokální jméno zmizí, objekt zůstává dostupný.

Opačná situace je nebezpečná: odkaz může zůstat, ačkoli objekt již zanikl. Pravidla jazyka určují, kdy objekt zaniká a které odkazy lze dál použít.

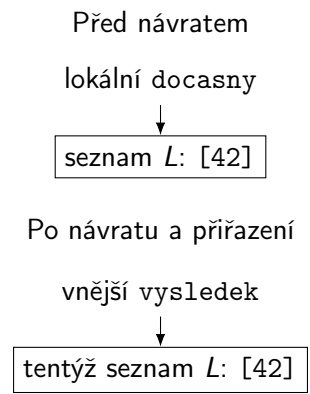
# Python: seznam přežije volání svého tvůrce

B2-N15, Python: `def` zavádí funkci, odsazení vymezuje její tělo.

```
def vytvor():
    docasny = [42]
    return docasny

vysledek = vytvor()
print(vysledek)
```

Výstup: `[42]`.




`return` vrátí referenci na seznam, `print` jej vypíše. Šipky vedou v obou časech k témuž objektu.

# Kontrola porozumění: co z čeho plyne?

1. Změní přiřazení hodnoty také deklaraci, ke které jméno patří?
2. Zaniká vnější objekt tím, že jej vnitřní deklarace zastíní?
3. Musí lokální jméno znamenat objekt žijící jen po dobu volání?
4. Změní volání z jiné funkce lexikální vazbu jména?

U každé odpovědi uveďte pravidlo nebo protipříklad z předchozího výkladu.

# Typ, hodnota a reprezentace

- **Hodnota** je konkrétní údaj: číslo 12, text "12", pravda.
- **Typ** vymezuje přípustné hodnoty a pravidla jejich použití: například čísla lze sčítat.
- **Reprezentace** je způsob uložení údaje v implementaci; například pomocí bitů (jednotek s hodnotou 0 nebo 1).

V Pythonu 3 znamená stejný znak + dvě různé operace:

| | | |
|---|---|---|
| `12 + 1` | $\longrightarrow$ `13` | číselný součet |
| `"12" + "1"` | $\longrightarrow$ `"121"` | spojení textů |

Uvozovky zde rozlišují zápis textu od zápisu čísla. Podobný výpis hodnot tedy ještě neznamená stejný typ.

# Kdy se kontrolují typy?

**Typová kontrola** ověřuje, zda typy použitých hodnot vyhovují pravidlům operace.

| | Statická kontrola | Dynamická kontrola |
|---|---|---|
| Kdy | Před vykonáním programu. | Při vykonání operace. |
| Z čeho vychází | Z typů a pravidel programu. | Z druhů aktuálních hodnot. |
| Nevykonaná větev | Může obsahovat odmítnutou typovou chybu. | Její operace se za běhu nekontroluje. |

- Statické a dynamické kontroly lze kombinovat.
- **Překlad versus interpretace je jiná otázka.**
- „Silné/slabé“ bez definice nestačí: určeme povolené převody, čas kontroly a chování při chybě.
- Ani úspěšná typová kontrola sama neověří záměr zadání.

# Tři různé důvody, proč program nevyhoví

Python 3: každý řádek posuzujeme **samostatně**. `[4]` je jednoprvkový seznam s indexem (pořadovým číslem) 0. **Výjimka** oznámí chybu a přeruší běžné pokračování.

| Výraz | Výsledek | Co je špatně |
|---|---|---|
| `4 + "2"` | `TypeError` | Kombinace druhů operandů není pro + přípustná. |
| `[4][3]` | `IndexError` | Index je celé číslo, ale položka s indexem 3 neexistuje. |
| `2 + 3` | `5` | Pokud chceme obsah obdélníku o stranách 2 a 3, měl být výsledek 6. |

- Typová kontrola a kontrola mezí odpovídají na různé otázky.
- Správný běh operací sám nepotvrzuje správné řešení zadání.

# Typ můžeme napsat, nebo jej odvodit

**Anotace** je výslovný zápis typu. **Odvozování typu** jej určí z výrazu a pravidel jazyka.

K06-07b, C++17: `auto` žádá odvození z počáteční hodnoty.

```
auto pocet = 3;      // odvozen typ int: cele cislo
// pocet = "tri";  // po odkomentovani chyba prekladu
```

K06-07a, Python 3: anotace sama při běhu typ nevynucuje.

```
x: int = 3          # zapsana anotace int
x = "tri"
print(x)            # tri
```

**Pointa:** zápis typu a čas jeho kontroly jsou dvě různé otázky. Pythonovou anotaci může kontrolovat samostatný statický analyzátor.

# Základní hodnoty mají konkrétní meze

- **Celá čísla:** pevný rozsah, nebo reprezentace rostoucí podle hodnoty? Co se stane při překročení rozsahu?
- V modelu neznaménkového 8bitového čísla je 256 hodnot: 0 až 255. Uložíme-li zbytek po dělení 256, z výsledku $255 + 1$ dostaneme 0.
- **Pohyblivá čárka:** konečný rozsah a přesnost. Desetinné 0,1 nemá konečný dvojkový rozvoj.
- **Bool:** pravda a nepravda (`true`, `false`); pravidla případných převodů určuje jazyk.
- **Absence hodnoty:** například Python `None`; není to neexistující jméno ani prázdný řetězec.

# Délka textu: počítáme znaky, body, nebo bajty?

K06-04, Python 3: `s` je „kůň“, `t` je `u` + kombinující kroužek. Zápis `\uXXXX` určuje kódový bod Unicode.

```
s = "k\u016f\u0148"
t = "u\u030a"
print(len(s), len(s.encode("utf-8")))  # 3 5
print(len(t), len(t.encode("utf-8")))  # 2 3
```

- **Kódový bod**: číselně označená pozice v Unicode. Python `len(str)` počítá tyto jednotky.
- **UTF-8** převádí text na bajty (osmibitové jednotky); `encode` provede převod a `len` spočte délku.
- **Grafémový shluk**: skupina bodů přibližující vnímaný znak. Zde celé `t` tvoří jeden shluk, ale dva body.
- `t[1]` vybere jen kroužek: indexování Pythonu jde po bodech.

# Kolekce: položku hledáme indexem, nebo klíčem

**Kolekce** seskupuje více položek. K06-06a, C++17, výřez z `main`:

```
std::array<int, 3> pole{2, 4, 6};
std::vector<int> seznam{2, 4};
seznam.push_back(6);
std::map<std::string, int> mapa{{"Eva", 2}};
```

- `std::` označuje standardní knihovnu; `int` celé číslo a `std::string` text. Údaje v `< >` určují podobu kolekce.
- `array<int, 3>`: tři čísla; `pole[1]` je 4. **Index** čísluje pozici od nuly.
- `vector<int>`: rostoucí posloupnost čísel; `push_back` přidá na konec 6, proto `seznam[2]` je 6.
- `map<string, int>`: textové klíče a číselné hodnoty; `mapa.at("Eva")` je 2. **Klíč** označuje hledanou položku.

# Pravidla přístupu: co když položka chybí?

C++17: `pole` i `seznam` mají tři prvky; `mapa` jen klíč `"Eva"`. **Výjimka** oznámí chybu a přeruší běžné pokračování, dokud ji program nezachytí.

| Operace | Výsledek a účinek |
|---|---|
| `pole.at(3)` | Výjimka `std::out_of_range`; platné indexy jsou 0, 1, 2. |
| `seznam.at(3)` | Stejná výjimka; možnost růstu nezaručuje existenci čtvrtého prvku. |
| `mapa.at("Jan")` | Stejná výjimka; chybějící klíč se nevytvoří. |
| `mapa["Jan"]` | Vloží nový klíč s hodnotou `int` 0 a vrátí přístup k této hodnotě. |

- Pevná délka `array`, růst `vectoru` a přístup do mapy jsou různá pravidla.
- U C++17 `array` a `vectoru` není `[]` kontrolovanou alternativou k `at`.

# N-tice, záznam a varianta

Jednu hodnotu můžeme také složit z více částí, nebo připustit několik různých podob.

| Tvar | Příklad v pseudokódu | Jak získáme část |
|---|---|---|
| N-tice | `("Eva", 2)` | Pozicí složky. |
| Záznam | `Bod{x: 2, y: 3}` | Jménem položky. |
| Varianta | `Hodnota(4)` nebo `Chyba("nula")` | Nejprve rozlišením případu. |

- N-tice a záznam obsahují své složky **současně**.
- Rozlišená varianta nese **jeden z přípustných případů**.
- Před číselnou operací musíme vědět, že máme `Hodnota`, ne `Chyba`.
- Tvar dat není totéž co způsob jejich fyzického uložení.

# Krátké srovnání: hodnoty hotového Mini

| Druh v Mini | Příklad | Alternativa v implementačním `Value` |
|---|---|---|
| Číslo | `12`, `2.5` | C++ `double` |
| Bool | `true`, `false` | C++ `bool` |
| Řetězec | `"Mini"` | C++ `std::string` |
| Nil | `nil` | `monostate`: prázdná alternativa |

- `mini::Value` je implementační C++ typ, který drží jednu z těchto alternativ; `double` je číselný typ s pohyblivou čárkou.
- Proměnná Mini může postupně nést různé druhy hodnot.
- To je implementace hotového Mini; zdrojové Mini nemá deklarace vlastních variant ani kolekce.

# Změna vazby a změna objektu

Python 3: **objekt** je konkrétní datový celek za běhu. **Mutabilita** znamená, že jeho obsah lze měnit. `[1]` vytvoří seznam.

```
a = [1]
b = a
b.append(2)
print(a)        # [1, 2]
b = [9]
print(a, b)     # [1, 2] [9]
```

- `b = a` vytvoří druhou cestu ke stejnému objektu.
- `append(2)` přidá prvek 2 do seznamu: to je **mutace**.
- `b = [9]` změní vazbu `b`; původní seznam nezmění.

# Mělká kopie stále může sdílet vnitřní data

K06-02a, Python 3: `[[1]]` je seznam obsahující další seznam. `copy()` vytvoří nový vnější seznam; `is` testuje, zda jde o **tentýž objekt**.

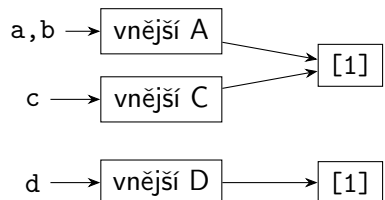
```
a = [[1]]
b = a
c = a.copy()
print(a is b, a is c, a[0] is c[0])
b[0].append(2)
print(a, c)
c.append([3])
print(a, c)
```

- Test totožnosti: `True False True`; a a c se liší, ale jejich první položka je společná.
- Po první změně: `[[1, 2]] [[1, 2]]`.
- Po druhé: `[[1, 2]] [[1, 2], [3]]`.

# Které seznamy kopie skutečně oddělí?

Python 3; nový běh. `from copy import deepcopy` zpřístupní funkci hluboké kopie; diagram je před změnou dat:

```
from copy import deepcopy
a = [[1]]
b = a
c = a.copy()
d = deepcopy(a)
```




- Mělká kopie `c` sdílí vnitřní seznam. Hluboká kopie `d` zde vytvoří i nový vnitřní seznam.
- Po `a[0].append(2)`: `a`, `b`, `c` jsou `[[1, 2]]`; `d` zůstane `[[1]]`.

# Přiřazení parametru není změna proměnné volajícího

K06-02b, Python 3: `def` zavádí funkci; odsazené řádky tvoří její tělo. `x` je místní parametr navázaný při volání na objekt `a`.

```
def prevaz(x):
    x = [[9]]
def zmen(x):
    x[0].append(4)
a = [[1]]
prevaz(a)
print(a)        # [[1]]
zmen(a)
print(a)        # [[1, 4]]
```

- `prevaz` přesměruje jen místní `x`.
- `zmen` upraví objekt dostupný i přes `a`.

# Totožnost typů, přípustnost použití a převod

| Otázka | Co se posuzuje |
|---|---|
| Ekvivalence | Jsou dva typové zápisy podle pravidel jazyka tentýž typ? |
| Kompatibilita | Smíme hodnotu použít právě v tomto kontextu? |
| Konverze | Jak se z hodnoty jednoho typu získá hodnota jiného typu? |

V C++17 je `double d = 3;` přípustné.

- Zdrojový výraz má typ `int`, cílová proměnná `double`.
- Proběhne dovolená konverze; oba typy zůstávají různé.
- Přípustnost použití může být směrová, není to totožnost.

# Podle jména, nebo podle struktury?

K06-03a, C++17: `struct` zavádí typ s položkami. `m.x` čte položku; `Bod{2, 3}` sestaví bod. `void ukaz(Bod p);` jen oznamuje funkci bez vracené hodnoty.

```
struct Bod   { int x; int y; };
struct Misto { int x; int y; };
void ukaz(Bod p);
// V tele main:
Misto m{2, 3};
// ukaz(m);           // chyba: ruzne typy
ukaz(Bod{m.x, m.y});
```

- **Ekvivalence podle jména**: zde mají `Bod` a `Misto` samostatné deklarace, a tedy různé C++ typy.
- **Strukturální ekvivalence:** při porovnání pouze jmen a typů položek mohou být tyto dva tvary ekvivalentní.
- Kritérium ekvivalence stanoví jazyk, nikoli podobný vzhled kódu.

# Převod může ztratit informaci

C++17, výřez z těla `main`:

```
double presne = 3;
int zkracene = static_cast<int>(3.75);
```

- **Implicitní** převod jazyk doplní podle pravidel: z čísla 3 vznikne přesné `double` 3.
- **Explicitní** převod žádáme zápisem: `static_cast<int>` znamená „převeď na `int`“; z 3.75 zde vznikne 3.
- Přípustný převod nemusí zachovat veškerou informaci.
- U převodů kontrolujeme také rozsah cíle a pravidlo zaokrouhlení.

Zápis převodu neříká, že původní typy jsou totožné.

# K06-03b: jedno jméno pro objem dvou těles

**Polymorfismus** umožňuje používat operaci pro více typů. **Přetěžování** nabízí pod jedním jménem konkrétní varianty.
C++17: při jedné délce počítáme objem krychle, při třech objem kvádru:

```
int volume(int s) { return s * s * s; }
long volume(long l, int b, int h) { return l * b * h; }
```

| Volání | Vybraná definice a výpočet | Výsledek |
|---|---|---|
| `volume(10)` | jeden argument: $10 \cdot 10 \cdot 10$ | 1000 |
| `volume(100L, 75, 15)` | tři argumenty: $100 \cdot 75 \cdot 15$ | 112500 |

- **Výběr proběhne při překladu**; zde rozhodne už počet argumentů. Funkce nemá uvnitř podmínku „krychle, nebo kvádr“.
- Typ před jménem je typ výsledku; `int s` je parametr `s` typu `int`. `long` je další celočíselný typ, přípona `L` jej žádá u literálu `100L`.

# Doplněk: stejný počet argumentů, jiný typ

C++17, vlastní příklad: obě funkce `popis` mají **jeden** parametr. `std::string` je typ textu; `double` je číselný typ s pohyblivou čárkou.

```
std::string popis(int x) { return "varianta int"; }
std::string popis(double x) { return "varianta double"; }
```

| Volání | Rozhodující typ argumentu | Vrácený text |
|---|---|---|
| `popis(3)` | `int` $\longrightarrow$ `popis(int)` | `varianta int` |
| `popis(3.0)` | `double` $\longrightarrow$ `popis(double)` | `varianta double` |

- Rozhodují statické typy při překladu; nikoli to, zda číselná hodnota právě obsahuje nenulovou desetinnou část.
- Samotný rozdíl *návratového* typu v C++ k přetížení nestačí.

# K06-03c: jeden předpis, místo typu je T

**Parametrický polymorfismus**: jeden předpis s parametrem typu. Zde vracíme dodanou hodnotu stejným postupem pro číslo i pravdivostní hodnotu. C++17:

```
template<class T>
T identita(T x) { return x; }
```

| Volání | Co se dosadí | Vrácená hodnota |
|---|---|---|
| `identita<int>(42)` | za `T` typ `int`; do `x` číslo 42 | 42 |
| `identita<bool>(true)` | za `T` typ `bool`; do `x` pravda | `true` |

- `template<class T>` zavádí **šablonu** s parametrem typu `T`; `class` zde nevyžaduje uživatelskou třídu.
- `T` není hodnota 42. Určuje typ parametru `x` i výsledku; konkrétní podobu zde vytvoří překladač.
- `x` je parametr pro hodnotu, kterou zpracuje volání za běhu.

# Doplněk: stejný postup pro číslo i text

C++17, vlastní příklad. Chceme dvakrát použít stejnou hodnotu pomocí +; pro číslo jde o součet, pro text o spojení.

```
template<class T>
T dvakrat(T x) { return x + x; }
```

| Volání | Použití téhož předpisu | Výsledek |
|---|---|---|
| `dvakrat<int>(4)` | `4 + 4` | 8 |
| `dvakrat<std::string>("ha")` | `x + x`: spojení textů | `"haha"` |

- `T` může zastupovat jen typ, pro který je tělo přípustné; zde potřebujeme použitelné + a návrat výsledku jako `T`.
- Napsali jsme **jedno tělo** funkce `dvakrat`. Předchozí přetěžování `popis` mělo **dvě samostatná těla**.

# K06-03d: různé objekty, společná operace

**Podtypový polymorfismus**: různé druhy objektů nabízejí společné **rozhraní** (operace, které smíme volat). **Třída** popisuje druh objektů; **metoda** jeho operaci. C++17: požádáme zvíře o zvuk.

```
struct Zvire {
    virtual std::string zvuk() = 0;
};
struct Pes : Zvire {
    std::string zvuk() override { return "haf"; }
};
```

- `Pes : Zvire`: Pes je zde **podtyp**; lze jej použít tam, kde se očekává `Zvire`.
- `virtual` umožňuje vybrat metodu podle skutečného objektu; `= 0` zde požaduje metodu bez těla v `Zvire`.
- `override` žádá kontrolu, že metoda přepisuje zděděnou virtuální metodu. Tělo pro psa vrací `"haf"`.

# Doplněk: stejná funkce osloví psa i kočku

K06-03d, pokračování. **Instance** je konkrétní objekt třídy: `Pes p;` vytvoří psa `p`.

```
struct Kocka : Zvire {
    std::string zvuk() override { return "mnau"; }
};
void ozvi_se(Zvire& x) { std::cout << x.zvuk() << '\n'; }
int main() {
    Pes p; Kocka k;
    ozvi_se(p); ozvi_se(k);
}
```

- `Zvire& x` je reference: zpřístupní předaný objekt přes společné rozhraní, nevytváří kopii zvířete.
- `x.zvuk()` volá metodu: u `p` se vybere `Pes::zvuk` → `haf`, u `k` `Kocka::zvuk` → `mnau`.
- Toto **virtuální volání** se řídí druhem objektu za běhu. Vybírá se metoda objektu, nikoli proměnná podle volajících jako v Bashi.

# Doplněk: proč jsme potřebovali všechny tři ukázky?

| Konkrétní volání | Co jsme napsali | Co se rozhodne |
|---|---|---|
| `volume(10)` | Dvě samostatná těla `volume`. | Při překladu se vybere varianta s jedním parametrem; dá 1000. |
| `identita<int>(42)` | Jeden předpis s parametrem typu `T`. | Překladač dosadí `int`; volání předá 42 a vrátí 42. |
| `ozvi_se(k)` | Jedna funkce přijímající rozhraní `Zvire`; konkrétní metody psa a kočky. | `x.zvuk()` zvolí podle skutečného objektu metodu kočky a vypíše `mnau`. |

**Odnesme si:** přetěžování vybírá *variantu funkce*; šablona doplňuje *typ do předpisu*; podtypové použití dovolí *společné rozhraní s různým chováním objektů*.

# Krátké srovnání: co zaručí Mini check?

K06-01, záměrná chyba; hotové Mini:

```
fun main() {
  print 2 + "3";
}
```

- `minic check`: úspěch, návratový kód 0 (značka úspěchu nástroje).
- `minic run`: běhová chyba, návratový kód 1 (značka neúspěchu).
- Jména a syntaxe jsou přípustné; + ale vyžaduje dvě čísla nebo dva řetězce.
- Překlad do C++ nezruší dynamickou kontrolu hodnot Mini.

# Zkuste rozhodnout a zdůvodnit

1. Znamená `auto x = 3;` v C++ dynamické typování?
2. Má „kůň“ v UTF-8 stejný počet bajtů a kódových bodů?
3. Zaručí `a.copy()` v Pythonu nezávislost všech vnořených seznamů?
4. Jsou dva typy totožné, pokud jazyk dovoluje mezi nimi převod?
5. Jak se liší výběr přetížené funkce, parametr typu a podtypové použití?

# Od textu přes vazby k hodnotám

1. Lexer a parser určují jednotky a strukturu programu.
2. Pravidla rozsahu spojují jména s deklaracemi.
3. Při běhu vznikají objekty a mění se hodnoty.
4. Typová pravidla určují přípustné operace a převody.

Domácí úkol: celý resolver v připraveném `block02-start`. Zadání, postup sestavení a testy jsou v samostatném materiálu.

Po hodině: kapitoly 4–6 a samostatný doplněk ke četbě.

Příště: kapitoly 7–9, vyhodnocování, řízení toku a funkce.

# Zdroje:


1. *Programovací jazyky*, kapitoly 4–6: zpracování programu, jména a životnost, typy a hodnoty. Převzatá ID K04–K06 jsou zachována; zkrácení ukázek jsou označena.
2. Robert W. Sebesta: *Concepts of Programming Languages*, 10. vydání, Pearson, 2012. Kap. 4–6; skriptování §2.18, funkce §9.1–9.2, polymorfismus §9.8–9.9.
3. Robert Nystrom: *Crafting Interpreters*. Kap. 2, 4–11 a 18: implementační cesty, scanning, AST, parsing, vazby a reprezentace hodnot.
4. ISO C++: pracovní návrh N4659 (C++17). Typy, rozsahy, životnost, převody, přetěžování, šablony, virtuální funkce a kontrakty kolekcí.
5. Luca Cardelli a Peter Wegner: *On Understanding Types, Data Abstraction, and Polymorphism*, 1985. §1.3, §4.1, §6.1: tři mechanismy polymorfismu.
6. Python 3.12: Data model, Built-in Types, copy, typing. Simple statements. Hodnoty, přiřazení, mutabilita, kopie, anotace a Unicode.

# Zdroje:


7. Unicode Consortium: *Unicode 17.0*, kap. 2; UAX #29, rev. 47, §3.1.1, GB9. Kódové body, UTF-8 a grafémové shluky (K06-04).
8. *Mini 1.0*: specifikace, architektura a finální implementace `Mini_2_0`. K04-01: *4W minilang intro*, snímek 59, beze změny. K05-01: zalomení a redukce D0/D1; K06-01: chybná varianta z kap. 6.
9. K06-03b: přispěvatelé Wikipedie, *Function overloading*, rev. 1376490484; autoři a sloučená historie. Kód pod CC BY-SA 4.0. Vynechán válec, výpis upraven pro C++17, odstraněny komentáře a změněno formátování; volání přesunuta do tabulky, `100l` změněno na `100L`.
10. GNU: *Bash Reference Manual*, Shell Functions: dynamický rozsah lokálních proměnných. C++/Bash je nová srovnávací varianta příkladu z week06.
11. Dodané výukové podklady *week05/session02*, *week06/session01–02* a *week07/session01*: syntaxe, vazby a typy. Stopy a schémata nově zpracovány; pravidla modelů jsou výslovně odlišena od Mini.
12. Ostatní výřezy K04–K06 jsou z kapitol; B2-T01 a doplňky F01, B01–B02, T01–T04 jsou nové varianty. K06-03d používá textový typ a dvě zvířata.